

Chapter 1: Introduction

1.1 Overview

Alopecia, the clinical term for pathological hair loss, affects individuals across a broad range of ages and demographic backgrounds worldwide. Some degree of hair shedding is a normal part of the follicular growth cycle, but a marked or sudden increase in shedding often points to an underlying disturbance in the body's systems, commonly hormonal imbalance, significant nutrient depletion, prolonged physical strain, or sustained psychological stress. Beyond its visible effects, noticeable hair loss frequently carries a real emotional cost, contributing to lowered self-confidence, elevated anxiety, and a reduced sense of overall well-being.

Determining an individual's likelihood of experiencing hair loss is inherently difficult because no single biomarker reliably predicts it. Risk instead emerges from the combined, non-linear interaction of physiological traits, everyday habits, environmental factors, and pre-existing health conditions. Elements such as diet quality, sleep regularity, scalp care practices, thyroid activity, iron levels, and hereditary predisposition all interact at once, making it impractical for clinicians or individuals to gauge overall risk through observation alone.

Healthcare systems today generate large volumes of patient data through Electronic Health Records (EHRs), capturing detailed clinical histories over time. A substantial share of this clinical data is inherently tabular in structure: laboratory results, vital signs, diagnostic codes, medication histories, and patient demographics are all naturally represented as structured rows and columns rather than as free text or medical imagery. Consequently, tabular data forms the backbone of a large fraction of real-world clinical prediction tasks, including disease risk stratification, treatment outcome prediction, and, as in this study, early screening for conditions such as hair loss. Machine learning offers a way to surface patterns within this dense, multi-source tabular data that would otherwise go unnoticed. Gradient boosting methods, such as CatBoost, have long been the standard tool for prediction on structured, tabular health data. More recently, however, tabular foundation models, transformer architectures pretrained to perform prediction directly from context without task-specific training, have begun to reshape this space. Prior Labs' TabPFN and Google's TabFM both represent this newer generation of zero-shot tabular models, offering predictions through a single forward pass rather than the conventional train-and-tune cycle.

A persistent limitation of both gradient boosting and transformer-based approaches, however, is their tendency to operate as opaque, uninterpretable systems. This research addressed that limitation by building a comparative machine learning framework for evaluating and explaining multi-tier hair fall risk, using structured clinical and lifestyle data. The study benchmarked three distinct modeling paradigms: CatBoost, a gradient-boosted decision tree architecture purpose-built for categorical data; TabPFN, an established prior-fitted transformer foundation model for tabular in-context learning; and TabFM, Google Research's more recently released zero-shot tabular foundation model, which combines row-and-column attention with an in-context learning transformer. Post-hoc explainability was layered across all three models via Shapley Additive Explanations (SHAP), giving each prediction a transparent, feature-level justification grounded in the underlying medical and lifestyle indicators.

1.2 Problem Statement

Determining hair loss risk remains a genuinely difficult clinical problem, shaped by the interplay of hormonal shifts, metabolic irregularities, chronic stress, nutritional gaps, and hair care habits. Current practice leans heavily on subjective self-reporting or delayed clinical evaluation, both of which are vulnerable to observer bias, inconsistent standards, and limited reach in under-resourced settings.

While computational methods can, in principle, model these interacting risk factors systematically, most existing dermatological prediction research still relies on older algorithms, such as Logistic Regression or standard Random Forests, that demand substantial manual feature engineering and struggle to capture complex feature interactions. Moreover, the dermatology literature had not yet meaningfully compared how newer zero-shot tabular foundation models perform against specialized gradient-boosted trees on real-world health survey data, and no existing work had extended this comparison to include more than one such foundation model architecture.

This study addressed these gaps directly by building and benchmarking an explainable machine learning framework for multi-tier hair fall risk classification. By evaluating CatBoost against two independently developed tabular foundation models, TabPFN and TabFM, alongside SHAP-based feature attribution, this research examined whether zero-shot, prior-fitted transformer approaches can match or exceed a tuned gradient-boosted tree architecture on tabular health data, while keeping every predicted risk tier traceable to specific, interpretable clinical indicators.

1.3 Objective

The objective of this dissertation is:

- To implement, benchmark, and explain a machine learning framework comparing CatBoost, TabPFN, and TabFM for predicting multi-tier hair fall risk from structured clinical and lifestyle data, while providing transparent, feature-level decision-support insights through SHAP-based explainability.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Alopecia progresses through a range of clinical stages, from mild, barely noticeable shedding to significant, visible scalp thinning. Conventional clinical assessment depends largely on subjective observation and periodic self-reporting from patients, an approach that tends to be inconsistent, susceptible to observer bias, and difficult to access in under-resourced or rural areas.

The clinical complexity of hair loss compounds this diagnostic difficulty. Hair fall risk arises from the interaction of multiple, often co-occurring physiological systems, including endocrine regulation, micronutrient status, hepatic function, and chronic psychological stress, rather than from any single dominant cause. A clinician evaluating a single patient must therefore integrate signals from several unrelated biological domains at once, a task that becomes harder as the number of relevant variables grows and that resists standardization across clinicians and settings. Existing dermatological screening practices have generally not kept pace with this complexity, continuing to rely on qualitative severity scales and patient-reported history rather than structured, quantitative risk models.

Machine learning offers a more systematic alternative. It can model the simultaneous influence of physiological, behavioral, and clinical variables on hair loss outcomes rather than relying on a single observable indicator or clinician judgment alone. Unlike traditional clinical heuristics, which typically weigh a small number of familiar risk factors, machine learning models can learn complex, non-linear interactions across a much larger set of variables directly from data, without requiring these interactions to be specified in advance. This suits conditions such as hair loss, where risk is multi-factorial and no single biomarker provides sufficient diagnostic signal on its own.

2.1.1 Theoretical Foundations of Tabular Machine Learning Architectures

This study drew on three theoretical paradigms in modern tabular machine learning: ensemble-based gradient boosting, tabular foundation models operating through in-context learning, and post-hoc explainability grounded in cooperative game theory. This section introduces the concepts behind each. Chapter 3 describes the specific models used in this study and their mathematical formulations.

Ensemble Learning and Gradient Boosting. Ensemble learning combines the predictions of multiple simpler models to produce a stronger, more robust prediction than any individual model alone. Boosting, a specific ensemble strategy, trains models sequentially, with each new model correcting the errors of the ensemble built so far. Gradient boosting frames this correction as a numerical optimization problem, fitting each new weak learner to the residual errors of the current ensemble. Because each tree targets residual error rather than the raw target, gradient boosting ensembles can capture non-linear feature interactions beyond what a single decision tree can represent. A recurring challenge in applying gradient boosting to tabular data is handling categorical variables, since naïve encoding approaches can introduce sparsity and target leakage. This motivated specialized encoding strategies in modern boosting frameworks.

Tabular Foundation Models and In-Context Learning. A more recent paradigm draws on the foundation model concept from large language models: a single model pretrained once, offline, on a large and diverse corpus, capable of generalizing to new tasks without retraining. Applied to tabular data, this is realized through in-context learning. A model is pretrained, typically on synthetic datasets generated from structural causal models, to learn tabular prediction as a general skill rather than dataset-specific parameters. At inference, the model is given a training context directly alongside a query and produces a prediction in a single forward pass, without gradient-based training. This is typically implemented through self-attention, the core building block of transformer architectures, which weighs how much each element of the input should attend to every other element. Different tabular foundation models vary in how they apply this attention, for instance across samples, across features, or both. These represent distinct design choices within the same paradigm, which motivates comparing more than one such architecture empirically rather than assuming results generalize across models.

Explainability via Cooperative Game Theory. Both gradient boosting ensembles and attention-based foundation models are difficult to interpret directly and offer no inherent explanation for individual predictions. This opacity is consequential in clinical applications, where a practitioner may reasonably require justification for a risk assessment before acting on it. Shapley Additive Explanations (SHAP) address this by drawing on Shapley values, a concept from cooperative game theory for fairly distributing a payoff among contributing players. Applied to machine learning, SHAP treats each feature as a player and the model's prediction as the payoff, providing a model-agnostic, theoretically grounded method for attributing predictions to specific input features. This property suits comparing explainability consistently across architecturally distinct models.

2.2 Literature Review

Machine learning approaches to hair loss prediction have grown steadily in recent years, drawing on a range of classical and modern algorithms applied to both structured survey data and scalp imagery. Widowati et al. [1] developed an explainable decision-support system for multi-class hair loss severity classification, emphasizing that restricting feature transformations to within cross-validation folds prevents inflated generalization estimates. Sayyad and Midhunchakkaravarthy [2] built a diagnostic model specifically targeting Alopecia Areata, improving prediction accuracy for that single condition, though their approach did not extend to the broader set of multi-factorial risk drivers relevant to hair loss generally.

Sai et al. [3] directly compared several individual classification algorithms against an ensemble method for hair fall prediction, finding that the ensemble approach consistently outperformed every individual algorithm tested across accuracy, precision, and recall. Patel et al. [4] proposed a Random Forest-based framework incorporating hereditary factors, hormonal markers, medical history, and nutritional status, and notably found that Random Forest outperformed several competing gradient-boosting approaches including XGBoost, CatBoost, and LightGBM on their dataset.

A separate deep learning investigation [5] combined genetic, hormonal, and scalp-condition data with lifestyle variables to compare standard classifiers against neural network architectures, noting that while neural models captured more complex feature interactions, this came at the cost of per-prediction interpretability.

Another applied study [6] examined survey-based lifestyle and clinical indicators, including dandruff severity, chronic illness history, thyroid function, and micronutrient status, using ROC-AUC and macro-F1 to account for class imbalance.

Recent dermatological and scalp-health classification studies [7] have shown CatBoost outperforming conventional level-wise tree algorithms on datasets dominated by categorical health survey features, a performance advantage attributed to CatBoost's Ordered Target Encoding and its resistance to target leakage.

The emergence of tabular foundation models such as TabPFN [8] has established a new benchmark for zero-shot clinical decision support, achieving competitive predictive performance without iterative hyperparameter tuning.

The broader clinical machine learning literature [9] stresses that predictive accuracy alone is insufficient without accompanying explainability, and that integrating SHAP-based feature attributions with modern tabular models allows practitioners to cross-check automated predictions against known biological mechanisms.

Google Research's TabFM [10] extends this paradigm through a hybrid attention architecture that alternates row- and column-wise processing before applying in-context prediction, and has been shown on the TabArena benchmark suite to outperform heavily tuned gradient-boosted baselines in a purely zero-shot setting.

Shakeel et al. [11] proposed an image-based classification framework distinguishing healthy hair from Alopecia Areata, extracting texture, shape, and color features from scalp images before applying Support Vector Machine (SVM) and k-Nearest Neighbor (KNN) classifiers, achieving 91.4% accuracy compared to 88.9% for KNN.

An independent third-party reproduction of TabFM [12], conducted across three separate machines and thirteen datasets, confirmed the model's competitive zero-shot accuracy against XGBoost, Random Forest, and TabPFN baselines, while also documenting four distinct upstream defects in the official implementation.

The physiological biomarker data for this study's dataset construction was sourced from a publicly available dataset [13], selected for its coverage of continuous clinical indicators relevant to hair health.

The demographic and lifestyle indicator data was sourced from a separate, publicly available survey dataset [14], selected for its coverage of binary lifestyle and hereditary risk factors.

Table 1: Comparative Analysis of Articles

Paper (study)	Methodology	Performance Insights	Identified gaps	Results
Widowati et al. [1]	Explainable decision-support system for multi-class hair loss severity	Fold-restricted feature transformations avoid inflated estimates	Foundation models not benchmarked	Explainable multi-class severity classification
Sayyad & Midhunchakkaravarthy [2]	ML diagnostic model for Alopecia Areata	Improved accuracy for a single condition	Multi-factorial risk drivers not covered	Improved Alopecia Areata prediction
Sai et al. [3]	Individual algorithms compared against an ensemble	Ensemble best on accuracy, precision, recall	Foundation models not evaluated	Ensemble outperformed all individual algorithms
Patel et al. [4]	Random Forest with hereditary, hormonal, medical, nutritional factors	Random Forest ahead of XGBoost, CatBoost, LightGBM	Result specific to their dataset	Random Forest best among tested models
Chen et al. [5]	Deep learning on genetic, hormonal, scalp, lifestyle data	Captured more complex feature interactions	Reduced per-prediction interpretability	Richer modeling, lower interpretability

Anderson & Roberts [6]	Survey-based lifestyle and clinical indicators	ROC-AUC and macro-F1 used for class imbalance	Foundation models not evaluated	Severity prediction from survey features
Prokhorenkova et al. [7]	CatBoost with Ordered Target Encoding	Resists target leakage	General-purpose, not hair-loss specific	Unbiased boosting on categorical features
Hollmann et al. [8]	TabPFN, in-context transformer	Competitive without tuning	Designed for small tabular problems	Zero-shot tabular classification
Lundberg & Lee [9]	SHAP, Shapley-value attribution	Model-agnostic, additive explanations	Costly to estimate for large-context models	Unified feature attribution
Kong & Das [10]	TabFM, alternating row and column attention	Beat tuned gradient-boosted baselines zero-shot on TabArena	Recently released, limited maturity	Competitive zero-shot performance
Shakeel et al. [11]	Image features with SVM and KNN	91.4% accuracy vs 88.9% for KNN	Image-based, not structured clinical data	Healthy vs Alopecia Areata classification
devYRPauli [12]	Independent reproduction of TabFM	Competitive vs XGBoost, Random Forest, TabPFN	Four upstream defects documented	Confirmed competitive accuracy

The numbering is serial, in order of first appearance, exactly as in your proposal: [1] to [12] for the papers, [13] and [14] for the two datasets. My earlier alphabetical numbering came from the syllabus rule, which your accepted proposal does not follow. The only wording change from your proposal is that the two dataset sentences now say "was sourced" instead of "will be sourced".

Chapter 3: Methodology

3.1 Research Framework

The methodology adopted in this research followed an end-to-end, multi-tier machine learning pipeline designed to train, evaluate, and interpret hair fall risk predictions. The overall workflow unfolded across four sequential stages: data ingestion and preprocessing, parallel model training and inference across three distinct architectures (CatBoost, TabPFN, and TabFM), quantitative performance evaluation, and post-hoc explainability through SHAP.

[Insert Figure 1 here]

Figure 1: Overall System Architecture Diagram

Raw patient records first passed through a preprocessing stage that removed non-informative identifier fields, encoded categorical attributes, and prepared the feature set for downstream modeling. The cleaned data then branched into three independent modeling paths, each corresponding to one of the benchmarked architectures: CatBoost, trained through iterative gradient-boosted decision trees; TabPFN, operating through zero-shot in-context inference; and TabFM, Google's more recently introduced tabular foundation model, which likewise performs zero-shot prediction but through an alternating row-and-column attention mechanism. Each model's predictions were then passed through a shared evaluation stage, where accuracy, precision, recall, F1-score, and ROC-AUC were computed under a consistent methodology to allow direct comparison across all three approaches. Finally, SHAP was applied uniformly across all three trained models, producing both global, cohort-level feature importance rankings and individual, patient-level explanations for specific predictions.

Only the tense changed ("follows" to "followed", "will first pass" to "first passed", and so on). The wording and paragraph order are otherwise your proposal's, and no citations were added, following Syllabus 1. Reuse the same Figure 1 image from your proposal.

Ready for 3.2 (Dataset Description) when you are.

Chapter 3: Methodology

3.2 Dataset Description

The dataset used in this study consisted of 21,606 patient records, combining demographic attributes, continuous physiological biomarkers, and binary lifestyle and clinical indicators. The target variable, hair fall risk, was represented as three ordinal severity tiers: Low, Moderate, and High.

The feature set was organized into four broad categories:

- Demographics and Heredity
 - age: patient age, in years
 - gender: patient sex (Female, Male, Other)
 - family_hair_fall_history: binary indicator (0 = No, 1 = Yes) of hereditary predisposition to hair loss
- Continuous Physiological and Serum Biomarkers
 - Nutritional and mineral markers: total_protein (serum total protein, g/dL), iron (serum iron, µg/dL), calcium (serum calcium, mg/dL), vitamin_d (serum 25-hydroxy vitamin D, ng/mL), manganese (whole blood manganese, µg/L)
 - Hair and hydration indices: total_keratine and hair_texture, both composite 0–100 scores reflecting hair condition rather than standard clinical laboratory measurements; body_water_content, expressed as a percentage
 - Hepatic and stress biomarkers: alt_liver (serum alanine aminotransferase, U/L), stress_level (modeled on the 0–40 Perceived Stress Scale)
- Clinical and Coexisting Conditions (binary flags: 0 = No, 1 = Yes)

- chronic_illness: presence of a pre-existing systemic medical condition
 - anemia: documented iron or hemoglobin deficiency
 - stress: self-reported experience of high psychological stress
4. Lifestyle and Environmental Exposures (binary flags: 0 = No, 1 = Yes)
- late_night_sleep: habitual late sleep timing or delayed circadian pattern
 - sleep_disturbance: interrupted sleep or insomnia
 - water_reason: regular exposure to hard or chemically treated water
 - chemical_use: exposure to hair dyes, relaxers, or other chemical hair treatments

The physiological biomarker data was sourced from a publicly available dataset [13], and the demographic and lifestyle indicator data was sourced from a separate publicly available survey dataset [14], with respondent names removed prior to processing. As the two sources originated from different respondent groups, feature values were adjusted using domain-informed relationships from dermatological literature to produce the final dataset used in this study.

Only the tense changed ("will consist" to "consisted", "will be sourced" to "was sourced", and so on). The feature list is your proposal's, and the 20 features match the 20 columns left after removing `id`, `full_name`, and the target `hair_fall` in your code. [13] and [14] match the numbering in your 2.2, and the Clickbait example also cites its dataset in Chapter 3.

Your Chapter 5 limitations say the dataset construction was "disclosed in Section 2.1". It is disclosed here, so change that to Section 3.2 when we reach Chapter 5.

Ready for 3.3 (Data Preprocessing) when you are.

3.3 Data Preprocessing

To ensure data integrity and consistent handling across all three modeling architectures, the following preprocessing steps were applied prior to training:

1. **Identifier Removal:** Non-informative identifier columns (`id` and `full_name`) were removed prior to training, as they carry no predictive value and risk introducing spurious associations if retained.
2. **Categorical Encoding:** The gender attribute was normalized and mapped into a numeric format (0 = Female, 1 = Male, 2 = Other) to accommodate models requiring numeric input. The target variable, `hair_fall`, was already represented as zero-indexed ordinal labels ($Y \in \{0, 1, 2\}$, corresponding to Low, Moderate, and High risk respectively), requiring no additional transformation.
3. **Missing Value Handling:** The dataset was verified to contain no missing values across any feature or target column, so no imputation was required.
4. **Stratified Train–Test Split:** The cleaned data was divided into training and testing partitions using a stratified 80/20 split with a fixed random seed (`random_state = 42`), yielding 17,284 training records and 4,322 test records with class proportions preserved in both partitions. The same partition was used for all three models.

3.4 Model Selection

Rather than relying on traditional baseline classifiers or simple ensemble voting schemes, this study benchmarked three distinct, state-of-the-art tabular modeling paradigms.

3.4.1 State-of-the-Art Categorical Boosting: CatBoost

CatBoost was selected for its specialized handling of heterogeneous tabular data. As a gradient boosting framework, it builds decision trees sequentially, each correcting the residual errors of the previous ensemble, minimizing a regularized objective at iteration t :

$$\mathcal{L}^{(t)} = \sum_i L(y_i, F_{t-1}(x_i) + h_t(x_i)) + \Omega(h_t)$$

where $L(\cdot)$ is the multi-class cross-entropy loss, F_{t-1} is the prior ensemble, h_t is the new tree, and $\Omega(h_t)$ penalizes complexity to control overfitting. CatBoost mitigates target leakage through Ordered Target Encoding and builds

Symmetric (Oblivious) Decision Trees, giving it strong resistance to overfitting on relatively small clinical datasets and making it a robust tuned baseline for this study.

3.4.2 Frontier Tabular Foundation Model: TabPFN (v3)

TabPFN was selected as a well-established tabular foundation model that performs zero-shot prediction through in-context learning. Pretrained offline on synthetic datasets generated from structural causal models, it requires no dataset-specific training. At inference, it is given a training context D_{train} and query x_{test} directly, computing the prediction in a single forward pass:

$$P(y_{\text{test}} \mid x_{\text{test}}, \mathcal{D}_{\text{train}}) = \int P(y_{\text{test}} \mid x_{\text{test}}, \theta), P(\theta \mid \mathcal{D}_{\text{train}}), d\theta$$

This is implemented via self-attention, computing contextual relationships across features and samples:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

enabling zero-shot inference without hyperparameter tuning or feature scaling.

3.4.3 Zero-Shot Tabular Foundation Model: TabFM

TabFM was selected as a second, independently developed foundation model, allowing comparison between two competing zero-shot architectures rather than just gradient boosting versus one foundation model. Released by Google Research, TabFM alternates row- and column-wise attention over the input table before an in-context transformer produces predictions, distinguishing it from TabPFN's primarily sample-wise attention. Like TabPFN, it requires no training or tuning:

$$P(y_{\text{test}} \mid x_{\text{test}}, \mathcal{D}_{\text{train}}) = \text{TabFM}(x_{\text{test}}, \mathcal{D}_{\text{train}})$$

Including TabFM allowed this study to test whether zero-shot performance gains generalize across architectures, or are specific to one model's design.

3.5 Experimental Setup and Hyperparameter Tuning

Model configurations reflected each architecture's nature: a tuned gradient-boosted model versus two zero-shot foundation models requiring no optimization.

- CatBoost Setup:** Hyperparameter tuning used grid search across tree depth ($\text{depth} \in \{4, 6, 8\}$) and learning rate ($\text{learning_rate} \in \{0.01, 0.03, 0.1\}$), with L2-regularized leaf weights ($\text{l2_leaf_reg} = 3.0$), a maximum of 1,000 boosting iterations, and early stopping after 50 rounds. The MultiClass loss function was used with TotalF1 as the evaluation metric and a fixed random seed of 42. The optimal configuration was selected via Stratified 5-Fold Cross-Validation on the training partition, based on macro-F1, and resulted in $\text{depth} = 4$ and $\text{learning_rate} = 0.1$ (best cross-validation macro-F1 = 0.7665). The final model was then trained on the full training partition using this configuration.
- TabPFN Setup:** As a pretrained foundation model, TabPFN required no tuning. The default configuration was used on a GPU with a fixed random seed of 42. The full training partition (17,284 records) was supplied as context, and predictions were generated in a single forward pass.
- TabFM Setup:** Consistent with its zero-shot design, TabFM likewise required no tuning. The PyTorch implementation of the TabFM v1.0.0 classification model was used with a single ensemble member ($n_{\text{estimators}} = 1$). The training partition was supplied as context, and predictions were generated through its row-and-column attention mechanism. Because TabFM inference is computationally expensive, test predictions were generated in batches of 50 records with results saved to a checkpoint file after each batch.

To examine data efficiency, all three models were additionally evaluated under reduced training contexts of 100, 500, and 2,000 rows, each drawn at random (seed = 42) from the training partition. This analysis is reported in Section 4.3.

Unlike CatBoost's extensive grid search, TabPFN and TabFM required no tuning at all, a deliberate asymmetry central to this study's research question: whether zero-shot foundation models can match a carefully tuned gradient-boosted baseline without comparable optimization effort.

3.6 Explainability Architecture (SHAP Integration)

[Insert Figure 2 here]

Figure 2: SHAP Explainability Workflow

To address the interpretability limitations common to both boosting and transformer-based architectures, post-hoc explainability was integrated across all three trained models using Shapley Additive Explanations (SHAP). Grounded in cooperative game theory, SHAP decomposes each model's prediction into additive per-feature contributions:

$$f(x) = \phi_0 + \sum_j \phi_j$$

where ϕ_0 is the baseline expected prediction and ϕ_j quantifies the specific contribution of feature j (for example, iron, stress_level, total_protein, or family_hair_fall_history) to the final predicted risk tier.

For CatBoost, SHAP values were computed using the TreeExplainer algorithm, which exploits the model's tree structure to compute exact Shapley values efficiently across the full test set. For TabPFN and TabFM, neither of which exposes an internal tree structure, SHAP values were computed using KernelExplainer, which estimates attributions by querying the model directly. As kernel-based estimation requires repeated forward passes through each model's full in-context attention mechanism, its computational cost scales with both context size and the number of samples explained. The foundation models' SHAP analyses were therefore conducted using a reduced training context, a small background sample of 15 records, 100 kernel evaluations per explained instance, and a random subset of test instances (30 for TabPFN and 20 for TabFM), rather than the full training set and complete test partition used for TreeExplainer. The reason KernelExplainer was used instead of the originally planned PermutationExplainer is described in Section 4.5.

Across all three models, this framework produced patient-level waterfall plots, showing how individual features shifted a specific prediction away from the baseline, and cohort-level global importance rankings, showing which features most consistently influenced predictions across the sampled instances. Beeswarm plots and waterfall plots were generated for the High risk class.

3.7 Tools and Technologies

This research was implemented using the following Python-based software stack:

- **Data Manipulation and Preprocessing:** Pandas and NumPy for data handling, and Scikit-learn for stratified train-test splitting, Stratified 5-Fold Cross-Validation, and standard evaluation metrics.
- **Tabular AI Engines:** The official CatBoost framework for gradient-boosted tree training; the TabPFN Python package for zero-shot tabular foundation model inference; and Google Research's TabFM package (PyTorch backend) for the second zero-shot tabular foundation model.
- **Explainable AI:** The official SHAP library, which was used to compute TreeExplainer-based attributions for CatBoost and KernelExplainer-based attributions for TabPFN and TabFM.
- **Statistical Analysis:** Statsmodels, which was used to compute McNemar's test for pairwise statistical significance testing between model predictions, and Wilson score confidence intervals for reported accuracy figures.
- **Visualization:** Matplotlib and Seaborn for confusion matrices, training curves, comparison charts, and SHAP plots.

3.8 Experimental Environment

All experiments were executed within cloud-based notebook environments to ensure consistent execution time and latency measurements across all three models. A GPU-accelerated environment was necessary to support the computational demands of TabPFN and TabFM and to keep inference times tractable.

Table 2: Experimental Hardware and Software Specifications

Category	Parameter	Specification
----------	-----------	---------------

Hardware	Computing Platform	Kaggle Notebooks and Google Colaboratory (cloud)
Hardware	GPU	[GPU model and memory]
Hardware	System Memory (RAM)	[RAM]
Software	Operating System	Linux (cloud runtime)
Software	Programming Language	Python [version]
Software	Acceleration Engine	PyTorch (CUDA)
Software	Core AI & ML Libraries	CatBoost, TabPFN, TabFM, SHAP, Scikit-learn, Statsmodels, Pandas, NumPy

3.9 Performance Evaluation Metrics

Model performance across the three ordinal risk categories (Low, Moderate, High) was evaluated using macro-averaged statistical metrics, applied consistently across all three models, alongside computational efficiency indicators. In the formulas below, TP_k, FP_k, and FN_k denote the true positives, false positives, and false negatives for class k, and N is the number of test records.

- **Accuracy:** Quantifies overall classification accuracy across all three risk tiers:

$$\text{Accuracy} = \frac{\sum_{k=0}^2 \text{TP}_k}{N}$$

- **Macro-Precision:** Evaluates prediction fidelity across classes equally, without weighting by class size:

$$\text{Precision}_{\text{Macro}} = \frac{1}{3} \sum_{k=0}^2 \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}$$

- **Macro-Recall:** Measures sensitivity across target levels equally:

$$\text{Recall}_{\text{Macro}} = \frac{1}{3} \sum_{k=0}^2 \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}$$

- **Macro-F1 Score:** Computes the F1-score of each class as the harmonic mean of that class's precision and recall, then averages these across the three classes:

$$\text{Macro-F1} = \frac{1}{3} \sum_{k=0}^2 \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$$

- **ROC-AUC:** Computed using a one-vs-rest strategy across the three risk classes, macro-averaged to summarize discriminative performance independent of class size.
- **Computational Latency:** Training or context-fitting wall-clock time (seconds) and per-sample inference latency (milliseconds/sample) were recorded for each model to evaluate deployment feasibility, given that CatBoost's training-then-predict paradigm differs fundamentally from TabPFN's and TabFM's in-context, fit-free inference approach.

- **Statistical Significance Testing:** Pairwise differences in prediction accuracy between models were assessed using McNemar's test with continuity correction on the held-out test set, based on the counts b and c of records that one model classified correctly and the other did not:
$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$
Ninety-five percent confidence intervals for each model's accuracy were computed via the Wilson score interval method, providing a statistically grounded basis for comparing model performance beyond raw metric values alone.

Chapter 4: Implementation and Result Analysis

4.1 Implementation

4.1.1 Implementation Tools

The models were implemented in Python using Kaggle Notebooks with GPU acceleration as the development environment. The primary libraries used were:

- pandas and numpy
- scikit-learn
- catboost
- tabpfn
- tabfm (PyTorch backend)
- shap
- statsmodels
- torch
- matplotlib and seaborn

4.1.2 Implementation Details

Implementation details include:

1. Data loading

The dataset was imported from a CSV file into a pandas DataFrame. Its shape, duplicate rows, missing values, and the distribution of the target variable hair_fall were inspected before any processing. A fixed random seed of 42 was set for reproducibility.

```
import os
```

```
import time
```

```
import warnings
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from itertools import combinations, product
```

```
from sklearn.model_selection import train_test_split, StratifiedKFold
```

```
from sklearn.metrics import (
```

```
    confusion_matrix, classification_report, accuracy_score,
```

```
    precision_score, recall_score, f1_score, roc_auc_score
```

```
)
```

```
warnings.filterwarnings("ignore")
```

```
RANDOM_STATE = 42
```

```
np.random.seed(RANDOM_STATE)
```

```
df = pd.read_csv("/kaggle/input/datasets/nirajanshahi/hairfall/data.csv")
```

```
print("Shape:", df.shape)
```

```
print("Duplicates:", df.duplicated().sum())
```

```
print("Missing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
```

```
print("\nhair_fall distribution:\n", df["hair_fall"].value_counts().sort_index())
```

Figure 3: Data Loading

2. Data preprocessing

The identifier columns `id` and `full_name` were dropped because they carry no predictive value. The gender attribute was mapped to numeric codes (Female = 0, Male = 1, Other = 2). The remaining 20 columns formed the feature matrix `X`, and `hair_fall` (Low = 0, Moderate = 1, High = 2) formed the target `y`.

```
df_model = df.drop(columns=["id", "full_name"]).copy()
```

```
gender_map = {"Female": 0, "F": 0, "Male": 1, "M": 1, "Other": 2, "O": 2}
```

```
df_model["gender"] = df_model["gender"].map(gender_map)
```

```
class_names = ["Low", "Moderate", "High"]
```

```
X = df_model.drop(columns=["hair_fall"])
```

```
y = df_model["hair_fall"]
```

```
print("Gender value counts:\n", df_model["gender"].value_counts())

print("\nX shape:", X.shape, "| y shape:", y.shape)

print("\nClass distribution:\n", y.value_counts().sort_index())
```

Figure 4: Data Preprocessing

3. Exploratory visualization (class distribution)

A bar chart of the number of records in each risk tier was generated to check class balance before training, since imbalance can affect model learning and the choice of evaluation metrics.

```
class_counts = y.value_counts().sort_index()

plt.figure(figsize=(7, 5))

ax = sns.barplot(x=class_names, y=class_counts.values, palette=["#4F94D4", "#E8A33D", "#D45A47"],
edgecolor="black")

for i, v in enumerate(class_counts.values):

    ax.text(i, v, f"{v:}", ha="center", va="bottom", fontweight="bold")

plt.xlabel("Hair Fall Risk Tier")

plt.ylabel("Number of Records")

plt.title("Class Distribution: Low vs Moderate vs High")

plt.tight_layout()

plt.savefig("class_distribution.png", dpi=150)

plt.show()
```

Figure 5: Exploratory Visualization (Class Distribution)

4. Train/test split

The data was divided into training and testing partitions using a stratified 80/20 split with `random_state = 42`, giving 17,284 training records and 4,322 test records with class proportions preserved in both. The same partition was used for all three models. A Stratified 5-Fold object was also created for hyperparameter tuning on the training partition.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)
```

```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

print("Train shape:", X_train.shape, "| Test shape:", X_test.shape)

print("\nTrain class balance:\n", y_train.value_counts(normalize=True).sort_index())

print("\nTest class balance:\n", y_test.value_counts(normalize=True).sort_index())

```

Figure 6: Train/Test Split

5. CatBoost tuning and training

A grid search over tree depth {4, 6, 8} and learning rate {0.01, 0.03, 0.1} was run with Stratified 5-Fold cross-validation on the training partition, scored by macro-F1. The best configuration (depth = 4, learning_rate = 0.1, cross-validation macro-F1 = 0.7665) was used to train the final model on the full training partition with the MultiClass loss, l2_leaf_reg = 3.0, up to 1,000 iterations, and early stopping after 50 rounds. Training time and per-sample inference time were recorded.

```

from catboost import CatBoostClassifier

param_grid = {"depth": [4, 6, 8], "learning_rate": [0.01, 0.03, 0.1]}

grid_rows = []

for depth, lr in product(param_grid["depth"], param_grid["learning_rate"]):

    fold_scores = []

    for tr_idx, va_idx in skf.split(X_train, y_train):

        fold_model = CatBoostClassifier(

            iterations=1000, depth=depth, learning_rate=lr,

            early_stopping_rounds=50, loss_function="MultiClass",

            eval_metric="TotalF1", random_seed=RANDOM_STATE,

            verbose=0, l2_leaf_reg=3.0

        )

        fold_model.fit(

            X_train.iloc[tr_idx], y_train.iloc[tr_idx],

            eval_set=(X_train.iloc[va_idx], y_train.iloc[va_idx]),

            use_best_model=True

        )

        fold_pred = fold_model.predict(X_train.iloc[va_idx]).flatten()

```

```

        fold_scores.append(f1_score(y_train.iloc[va_idx], fold_pred, average="macro"))

    grid_rows.append({"depth": depth, "learning_rate": lr, "cv_macro_f1": np.mean(fold_scores)})

grid_results = pd.DataFrame(grid_rows).sort_values("cv_macro_f1", ascending=False).reset_index(drop=True)

print(grid_results.to_string(index=False))

best_depth = int(grid_results.loc[0, "depth"])
best_lr = float(grid_results.loc[0, "learning_rate"])

print(f"\nBest: depth={best_depth}, learning_rate={best_lr}, CV macro-F1={grid_results.loc[0, 'cv_macro_f1']:.4f}")

cb_model = CatBoostClassifier(
    iterations=1000, depth=best_depth, learning_rate=best_lr,
    early_stopping_rounds=50, loss_function="MultiClass",
    eval_metric="TotalF1", random_seed=RANDOM_STATE,
    verbose=0, l2_leaf_reg=3.0
)

t0 = time.time()

cb_model.fit(X_train, y_train, eval_set=(X_test, y_test), use_best_model=True)

cb_train_time = time.time() - t0

t0 = time.time()

y_pred_cb = cb_model.predict(X_test).flatten()

cb_predict_time = time.time() - t0

y_proba_cb = cb_model.predict_proba(X_test)

print(f"Training time: {cb_train_time:.2f}s | Inference: {(cb_predict_time / len(X_test)) * 1000:.4f} ms/sample")

```

Figure 7: CatBoost Tuning and Training

6. TabPFN inference

TabPFN was run on the GPU with `random_state = 42` and its default configuration. The license token was read from the Kaggle secrets store, and PyTorch's checkpoint loading was patched so the pretrained weights could be loaded. The full 17,284-record training partition was supplied as context through `fit`, and predictions and class

probabilities were generated for the test set. No tuning was performed.

```
import torch

from kaggle_secrets import UserSecretsClient

os.environ["TABPFN_TOKEN"] = UserSecretsClient().get_secret("TABPFN_TOKEN")

_original_torch_load = torch.load

def _patched_torch_load(*args, **kwargs):
    kwargs["weights_only"] = False
    return _original_torch_load(*args, **kwargs)

torch.load = _patched_torch_load

from tabpfn import TabPFNClassifier

tabpfn_model = TabPFNClassifier(device="cuda", random_state=RANDOM_STATE)

t0 = time.time()

tabpfn_model.fit(X_train, y_train)

tabpfn_fit_time = time.time() - t0

t0 = time.time()

y_pred_tabpfn = tabpfn_model.predict(X_test)

tabpfn_predict_time = time.time() - t0

y_proba_tabpfn = tabpfn_model.predict_proba(X_test)

print(f"Fit time: {tabpfn_fit_time:.2f}s | Inference: {(tabpfn_predict_time / len(X_test)) * 1000:.4f} ms/sample")
```

Figure 8: TabPFN Inference

7. TabFM inference

The PyTorch implementation of the TabFM v1.0.0 classification model was loaded, with the JAX backend disabled to force the PyTorch fallback. Because TabFM inference time grows steeply with context size, a 500-row training context was drawn at random (seed = 42) from the training partition, and this context was supplied to the model with a single ensemble member (n_estimators = 1). Predictions and class probabilities were then generated for the full test set of 4,322 records.

```
import sys

sys.modules["flax.nnx"] = None

from tabfm import TabFMClassifier

from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1_0_0

tabfm_base = tabfm_v1_0_0.load(model_type="classification")

np.random.seed(RANDOM_STATE)

context_idx = np.random.choice(X_train.index, size=500, replace=False)

X_train_context = X_train.loc[context_idx]

y_train_context = y_train.loc[context_idx]

tabfm_model = TabFMClassifier(model=tabfm_base, n_estimators=1)

t0 = time.time()

tabfm_model.fit(X_train_context, y_train_context)

tabfm_fit_time = time.time() - t0

t0 = time.time()

y_pred_tabfm = tabfm_model.predict(X_test)

tabfm_predict_time = time.time() - t0

y_proba_tabfm = tabfm_model.predict_proba(X_test)

print(f"Fit time: {tabfm_fit_time:.2f}s")

print(f"Predict time: {tabfm_predict_time / 60:.1f} min | Per-sample: {(tabfm_predict_time / len(X_test)) * 1000:.2f} ms")
```

8. Evaluation

For each model, accuracy, macro-precision, macro-recall, macro-F1, and one-vs-rest macro ROC-AUC were computed with scikit-learn, together with the per-class classification report and a confusion matrix saved as an image.

```
def evaluate_model(name, y_true, y_pred, y_proba, cmap, filename):

    y_true = np.asarray(y_true)

    y_pred = np.asarray(y_pred)

    metrics = {

        "Model": name,

        "Accuracy": accuracy_score(y_true, y_pred),

        "Macro_Precision": precision_score(y_true, y_pred, average="macro"),

        "Macro_Recall": recall_score(y_true, y_pred, average="macro"),

        "Macro_F1": f1_score(y_true, y_pred, average="macro"),

        "ROC_AUC": roc_auc_score(y_true, y_proba, multi_class="ovr", average="macro"),

    }

    print(f"\n{name}")

    print(classification_report(y_true, y_pred, target_names=class_names))

    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(6, 5))

    sns.heatmap(cm, annot=True, fmt="d", cmap=cmap, xticklabels=class_names, yticklabels=class_names)

    plt.xlabel("Predicted")

    plt.ylabel("Actual")

    plt.title(f"{name} — Confusion Matrix")

    plt.tight_layout()

    plt.savefig(filename, dpi=150)

    plt.show()

    return metrics

results_table = pd.DataFrame([

    evaluate_model("CatBoost", y_test, y_pred_cb, y_proba_cb, "Blues", "catboost_confusion_matrix.png"),

    evaluate_model("TabPFN", y_test, y_pred_tabpfn, y_proba_tabpfn, "Greens", "tabpfn_confusion_matrix.png"),
```

```

evaluate_model("TabFM", y_test, y_pred_tabfm, y_proba_tabfm, "Oranges", "tabfm_confusion_matrix.png"),
])

results_table.insert(1, "Training_Context", [len(X_train), len(X_train), len(X_train_context)])

print(results_table.to_string(index=False))

results_table.to_csv("final_three_way_results.csv", index=False)

```

Figure 10: Model Evaluation

9. Statistical significance testing

McNemar's test with continuity correction was applied to each pair of models. For each pair, the test records were cross-tabulated by whether each model classified them correctly, and the resulting 2×2 table was passed to statsmodels.

```

from statsmodels.stats.contingency_tables import mcnemar

y_true_values = y_test.values

correct = {

    "CatBoost": np.asarray(y_pred_cb) == y_true_values,

    "TabPFN": np.asarray(y_pred_tabpfm) == y_true_values,

    "TabFM": np.asarray(y_pred_tabfm) == y_true_values,

}

mcnemar_rows = []

for name_a, name_b in combinations(correct.keys(), 2):

    a, b = correct[name_a], correct[name_b]

    table = [

        [int(np.sum(a & b)), int(np.sum(a & ~b))],

        [int(np.sum(~a & b)), int(np.sum(~a & ~b))],

    ]

    outcome = mcnemar(table, exact=False, correction=True)

    mcnemar_rows.append({

        "Comparison": f"{name_a} vs {name_b}",

        "McNemar_Statistic": outcome.statistic,

```

```
"p_value": outcome.pvalue,
"Significant": outcome.pvalue < 0.05,
})
```

```
statistical_results = pd.DataFrame(mcnemar_rows)
print(statistical_results.to_string(index=False))
statistical_results.to_csv("final_statistical_analysis.csv", index=False)
```

Figure 11: McNemar's Test Computation

The implementation of the training-context scaling experiment and the SHAP explainability analysis is described in Sections 4.3 and 4.4.

Before you paste this:

- **Grid search:** the cell in step 5 is written from the procedure you described. Your pasted notebook only had the final CatBoost cell, so compare it with your actual grid-search cell and use your version for the screenshot if it differs.
- **Test set in early stopping:** the final CatBoost fit passes `X_test` as `eval_set`, exactly as in your run, so the test data influenced the number of trees. State this in Limitations, or re-run with a validation split from the training data.
- **Figure numbers:** Figures run from 3 to 11, and Chapter 4 continues as Figure 12 onward. The class distribution chart is Figure 5, so use the image from the code above.

Your zip matches the numbers I used. I checked Tables 3 to 5 against `Step_05` to `Step_10`, and I read all three confusion matrices, so I could add the error counts too.

4.2 Result Analysis

1. Model Evaluation

CatBoost, TabPFN, and TabFM were evaluated on the same held-out test partition of 4,322 records (1,945 Low, 1,513 Moderate, and 864 High). CatBoost and TabPFN used the full 17,284-record training partition, while TabFM used a 500-row training context. All three models scored far above the random-chance accuracy of 33.3% for a three-class problem, which shows that the feature set carries real predictive signal for hair fall risk. TabPFN reached the highest accuracy (78.57%), followed by CatBoost (78.51%) and TabFM (77.88%). TabPFN achieved this with no hyperparameter tuning, whereas CatBoost required a grid search. TabFM came within 0.69 percentage points of TabPFN using under 3% of the training records.

2. Confusion Matrices

The confusion matrices show, for each model, how many records of each true risk tier were assigned to each predicted tier. In all three models the correct predictions concentrate on the diagonal, and almost all errors fall between adjacent tiers. Only 3 records for CatBoost, 1 for TabPFN, and 3 for TabFM were confused between Low and High, out of 929, 926, and 956 misclassified records respectively. The High tier had the lowest recall in each model (0.70, 0.73, and 0.69), with 256, 230, and 269 of its 864 records predicted as Moderate.

[Insert Figure 12: CatBoost confusion matrix]

Figure 12: CatBoost Confusion Matrix

[Insert Figure 13: TabPFN confusion matrix]

Figure 13: TabPFN Confusion Matrix

[Insert Figure 14: TabFM confusion matrix]

Figure 14: TabFM Confusion Matrix

3. Performance Evaluation Metrics

Table 3: Model Performance Comparison

Model	Training Context	Accuracy	Macro-Precision	Macro-Recall	Macro-F1	ROC-AUC
CatBoost	17,284	78.51%	0.7915	0.7662	0.7763	0.9189
TabPFN	17,284	78.57%	0.7875	0.7722	0.7787	0.9232
TabFM	500	77.88%	0.7904	0.7608	0.7715	0.9180

4. TabPFN led on accuracy, macro-recall, macro-F1, and ROC-AUC, while CatBoost led on macro-precision. The largest gap between any two models was 0.0072 on macro-F1 and 0.0052 on ROC-AUC.

5. Per-Class Performance

Table 4: Per-Class F1 Scores

Model	Low	Moderate	High
CatBoost	0.86	0.71	0.77
TabPFN	0.86	0.71	0.78
TabFM	0.85	0.71	0.76

6. All three models classified the Low and High tiers more reliably than the Moderate tier. This is consistent with the ordinal structure of the target: the Moderate tier borders both other tiers, so it shares a decision boundary with each, while Low and High each border only one.

7. Statistical Significance

McNemar's test with continuity correction was applied to each pair of models on the same 4,322 test records.

Table 5: Pairwise McNemar's Test Results

Comparison	Statistic	p-value	Significant ($\alpha = 0.05$)
CatBoost vs TabPFN	0.0147	0.9037	No
CatBoost vs TabFM	1.6528	0.1986	No
TabPFN vs TabFM	2.5798	0.1082	No

8.

None of the three pairwise differences reached significance at $\alpha = 0.05$, so no significant difference in accuracy was detected between the models. A tuned gradient-boosted model, a zero-shot model with the full training data, and a zero-shot model with a 500-row context therefore performed comparably on this task. Because TabFM used a smaller context than the other two, a comparison with all three models on identical contexts is given in Section 4.3.

-
- **Loss curve.** Your zip also has `catboost_training_curve.png` in `Step_05_CatBoost`. The Clickbait example doesn't have one, but you can add it as Figure 15 at the end of item 1 if you want.
 - **Where the numbers come from.** Every value above is copied from your `output.txt` files, and the confusion counts are read from the three PNGs.
 - **Wilson intervals.** No file contains Wilson confidence intervals, though Chapter 3 says they were computed. Either run the snippet from my earlier message and add a column to Table 3, or delete that sentence from 3.7 and 3.9.
 - **Chapter 3.5.** The TabFM Setup bullet there still needs rewriting for the 500-row context.

Ready for 4.3 (Training Context Scaling) next. I'll read `context_scaling_comparison.csv` and `matched_2000ctx_results.csv` from the zip, so you don't need to send anything.

Your zip has all the numbers for 4.3. Two facts from the files change how it must be written, and I list them after the section.

4.3 Training Context Scaling

To examine how much each model depends on the volume of training data, CatBoost, TabPFN, and TabFM were retrained under reduced training contexts of 100, 500, and 2,000 rows. Each context was drawn at random (seed = 42) from the 17,284-record training partition. CatBoost was trained on the sampled rows with the same configuration as in Section 4.1, while TabPFN and TabFM received the sampled rows as their in-context training data. The 100-row and 500-row runs were evaluated on the full test partition of 4,322 records. Because TabFM inference time grows steeply with context size, the 2,000-row run was evaluated on a fixed random subset of 500 test records for all three models.

```
from catboost import CatBoostClassifier
```

```

from tabPFN import TabPFNCClassifier

from tabFM import TabFMClassifier

np.random.seed(RANDOM_STATE)

subset_idx = np.random.choice(X_test.index, size=500, replace=False)

X_test_500 = X_test.loc[subset_idx]

y_test_500 = y_test.loc[subset_idx]

eval_sets = {

    100: (X_test, y_test),

    500: (X_test, y_test),

    2000: (X_test_500, y_test_500),

}

def sample_context(size):

    np.random.seed(RANDOM_STATE)

    idx = np.random.choice(X_train.index, size=size, replace=False)

    return X_train.loc[idx], y_train.loc[idx]

def run_context(size):

    X_ctx, y_ctx = sample_context(size)

    X_eval, y_eval = eval_sets[size]

    rows = []

    cb = CatBoostClassifier(

        iterations=1000, depth=best_depth, learning_rate=best_lr,

        early_stopping_rounds=50, loss_function="MultiClass",

        eval_metric="TotalF1", random_seed=RANDOM_STATE,

        verbose=0, l2_leaf_reg=3.0

    )

    cb.fit(X_ctx, y_ctx, eval_set=(X_eval, y_eval), use_best_model=True)

    pred = cb.predict(X_eval).flatten()

```

```

rows.append(("CatBoost", pred))

pfn = TabPFNCClassifier(device="cuda", random_state=RANDOM_STATE)

pfn.fit(X_ctx, y_ctx)

rows.append(("TabPFN", pfn.predict(X_eval)))

fm = TabFMClassifier(model=tabfm_base, n_estimators=1)

fm.fit(X_ctx, y_ctx)

rows.append(("TabFM", fm.predict(X_eval)))

return [
    {
        "Context_Size": size,
        "Model": name,
        "Test_Set_Size": len(y_eval),
        "Accuracy": accuracy_score(y_eval, p),
        "Macro_F1": f1_score(y_eval, p, average="macro"),
    }
    for name, p in rows
]

scaling_rows = []

for size in [100, 500, 2000]:

    t0 = time.time()

    scaling_rows.extend(run_context(size))

    print(f"Context {size} done in {(time.time() - t0) / 60:.1f} min")

scaling_results = pd.DataFrame(scaling_rows)

print(scaling_results.to_string(index=False))

scaling_results.to_csv("context_scaling_results.csv", index=False)

```

Figure 15: Training Context Scaling Experiment

```

plot_df = scaling_results.pivot(index="Context_Size", columns="Model", values="Accuracy").sort_index()

x_labels = [str(i) for i in plot_df.index]

plt.figure(figsize=(8, 5))

for model_name in ["CatBoost", "TabPFN", "TabFM"]:

    plt.plot(x_labels, plot_df[model_name] * 100, marker="o", label=model_name)

plt.xlabel("Training Context Size (rows)")

plt.ylabel("Accuracy (%)")

plt.title("Accuracy vs Training Context Size")

plt.legend()

plt.grid(alpha=0.3)

plt.tight_layout()

plt.savefig("context_scaling_accuracy.png", dpi=150)

plt.show()

```

Figure 16: Accuracy Under Reduced Training Context

Table 6: Accuracy and Macro-F1 Under Reduced Training Context

Context Size	Test Records	CatBoost Acc.	TabPFN Acc.	TabFM Acc.	CatBoost F1	TabPFN F1	TabFM F1
100 rows	4,322	60.53%	71.54%	74.41%	0.5745	0.7062	0.7363
500 rows	4,322	69.64%	77.65%	77.88%	0.6768	0.7691	0.7715
2,000 rows	500	74.40%	76.20%	77.40%	0.7260	0.7508	0.7631
Full (17,284 rows)	4,322	78.51%	78.57%	—	0.7763	0.7787	—

At every reduced context size, both foundation models scored higher than CatBoost. At 100 rows, which is under 0.6% of the training partition, CatBoost reached 60.53% accuracy, trailing TabPFN by 11.01 points and TabFM by 13.88 points. At 500 rows the gaps were 8.01 and 8.24 points, and at 2,000 rows they were 1.80 and 3.00 points. The gap to CatBoost therefore narrowed as the context grew, and at full context CatBoost matched TabPFN (78.51% against 78.57%).

The foundation models retained most of their full-data performance from small contexts. With 500 rows, TabPFN was 0.92 points below its own full-context accuracy, and TabFM at 500 rows was 0.63 points below CatBoost's full-context accuracy,

using 2.9% of the training records. CatBoost, in contrast, was 8.87 points below its full-context accuracy at 500 rows. Between 100 and 500 rows, CatBoost gained 9.11 points, TabPFN 6.11, and TabFM 3.47, which shows that the tuned gradient-boosted model depended most heavily on training volume.

TabFM scored higher than TabPFN at all three reduced contexts, but the margin shrank from 2.87 points at 100 rows to 0.23 at 500 rows. The 2,000-row margin of 1.20 points was measured on only 500 test records, where the 95% interval for an accuracy near 76% is roughly ± 3.7 points, so the ordering of the two foundation models at that size cannot be treated as a reliable difference. McNemar's test was not run at the reduced contexts, so no significance claim is made for any of these comparisons.

Two facts from your files shape the wording:

1. **Test sets differ.** The 100-row and 500-row accuracies are exact multiples of $1/4,322$, so they used the full test set. The 2,000-row run used a 500-record subset. I stated this in the section and did not compare the 2,000-row column against the others in absolute terms. In particular, TabPFN's 76.20% at 2,000 rows is lower than its 77.65% at 500 rows only because the test sets differ. Your old Chapter 4 draft said TabFM beat TabPFN at 2,000 rows as a finding. That isn't supportable on 500 records, so I dropped it, and Chapter 5's "Investigating TabFM's advantage at 2,000 rows" future-work item should be removed or reworded.
2. **Missing 100-row code.** The zip has the 500-row and 2,000-row scripts but not the 100-row one. My code block above merges them into one procedure. Use your original cells for the screenshot if they differ, and confirm that the 100-row run also used `np.random.choice` with seed 42.

Also:

- **Chapter 3.5.** It says the reduced-context evaluation is reported in Section 4.3. Add one sentence there saying the 2,000-row run used a 500-record test subset.
- **Optional fix.** Re-running 2,000 rows on the full test set would remove the caveat. At your earlier 1.85 s per sample, that takes about 2.2 hours for TabFM alone.
- **Full-context TabFM.** The table leaves it blank ("—"), since that run never completed.
- **CatBoost early stopping.** It still passes the evaluation set as `eval_set`, so the Limitations note applies here too.

Ready for 4.4 (Explainability with SHAP) when you are. I'll read the three global importance plots from your zip.

All three SHAP plots and code are in your zip. Note that `code.py` in Steps 8a to 8c only holds two-line stubs, so I rebuilt the full SHAP cells from your notebook. The one gap is the 800-row TabPFN context, which I reconstructed from the Step 8b output line, so check it (see the notes below).

4.4 Explainability Analysis (SHAP)

4.4.1 SHAP Implementation

SHAP was applied to all three models to identify which clinical and lifestyle features drove the predicted risk tiers. For CatBoost, exact Shapley values were computed with TreeExplainer on the full test partition. TabPFN and TabFM do not expose a tree structure, so KernelExplainer was used, which estimates attributions by repeatedly querying the model. Because each query passes through the full in-context attention mechanism, these runs used a reduced training context, a background sample of 15 records, 100 kernel evaluations per explained record,

and a random subset of the test partition (30 records for TabPFN and 20 for TabFM). Global importance plots, beeswarm plots, and patient-level waterfall plots were produced, with the beeswarm and waterfall plots drawn for the High risk class (class index 2).

1. CatBoost (TreeExplainer)

```
import shap

high_class_idx = 2

explainer_cb = shap.TreeExplainer(cb_model)
shap_values_cb = explainer_cb.shap_values(X_test)

shap.summary_plot(shap_values_cb, X_test, class_names=class_names, plot_type="bar", show=False)
plt.title("CatBoost — Global Feature Importance (SHAP)")
plt.tight_layout()
plt.savefig("catboost_shap_global_importance.png", dpi=150, bbox_inches="tight")
plt.show()

if isinstance(shap_values_cb, list):
    shap_vals_high_cb = shap_values_cb[high_class_idx]
else:
    shap_vals_high_cb = shap_values_cb[:, :, high_class_idx]

if isinstance(explainer_cb.expected_value, (list, np.ndarray)):
    base_value_cb = explainer_cb.expected_value[high_class_idx]
else:
    base_value_cb = explainer_cb.expected_value

def save_beeswarm_and_waterfall(prefix, title, values_high, data, base_value):
    shap.summary_plot(values_high, data, show=False)
    plt.title(f"{title} — SHAP Beeswarm (High Risk)")
    plt.tight_layout()
    plt.savefig(f"{prefix}_shap_beeswarm_high.png", dpi=150, bbox_inches="tight")
    plt.show()

    explanation = shap.Explanation(
        values=values_high[0],
        base_values=base_value,
        data=data.iloc[0],
        feature_names=data.columns.tolist()
    )
    shap.plots.waterfall(explanation, show=False)
    plt.title(f"{title} — Patient Waterfall")
    plt.tight_layout()
    plt.savefig(f"{prefix}_shap_waterfall_patient.png", dpi=150, bbox_inches="tight")
    plt.show()

save_beeswarm_and_waterfall("catboost", "CatBoost", shap_vals_high_cb, X_test, base_value_cb)
```

Figure 17: CatBoost SHAP Computation

2. TabPFN (KernelExplainer)

```
np.random.seed(RANDOM_STATE)
small_idx = np.random.choice(X_train.index, size=800, replace=False)
X_train_small = X_train.loc[small_idx]
```

```

y_train_small = y_train.loc[small_idx]

tabpfnclassifier = TabPFNClassifier(device="cuda", random_state=RANDOM_STATE)
tabpfnclassifier.fit(X_train_small, y_train_small)

def tabpfnclassifier_predict_proba(X):
    return tabpfnclassifier.predict_proba(pd.DataFrame(X, columns=X_test.columns))

background_small = shap.sample(X_train_small, 15, random_state=RANDOM_STATE)
explainer_tabpfnclassifier = shap.KernelExplainer(tabpfnclassifier_predict_proba, background_small)

np.random.seed(RANDOM_STATE)
sample_idx_kernel = np.random.choice(X_test.index, size=30, replace=False)
X_test_sample_kernel = X_test.loc[sample_idx_kernel].reset_index(drop=True)

t0 = time.time()
shap_values_tabpfnclassifier = np.array(explainer_tabpfnclassifier.shap_values(X_test_sample_kernel, nsamples=100))
print(f"TabPFN SHAP time: {time.time() - t0:.1f}s | shape: {shap_values_tabpfnclassifier.shape}")

shap.summary_plot(shap_values_tabpfnclassifier[:, :, high_class_idx], X_test_sample_kernel, plot_type="bar",
show=False)
plt.title("TabPFN — Global Feature Importance (SHAP)")
plt.tight_layout()
plt.savefig("tabpfnclassifier_shap_global_importance.png", dpi=150, bbox_inches="tight")
plt.show()

save_beeswarm_and_waterfall(
    "tabpfnclassifier", "TabPFN",
    shap_values_tabpfnclassifier[:, :, high_class_idx],
    X_test_sample_kernel,
    explainer_tabpfnclassifier.expected_value[high_class_idx]
)

```

Figure 18: TabPFN SHAP Computation

3. TabFM (KernelExplainer)

TabFM's `predict_proba` failed when called with a single row, so single-row queries were duplicated to a two-row batch and only the first output row was kept.

```

def tabfm_predict_proba_padded(X):
    X_df = pd.DataFrame(X, columns=X_test.columns)
    if len(X_df) == 1:
        X_padded = pd.concat([X_df, X_df], ignore_index=True)
        return tabfm_model.predict_proba(X_padded)[:1]
    return tabfm_model.predict_proba(X_df)

background_tabfm = shap.sample(X_train_context, 15, random_state=RANDOM_STATE)
explainer_tabfm = shap.KernelExplainer(tabfm_predict_proba_padded, background_tabfm)

np.random.seed(RANDOM_STATE)
sample_idx_tabfm = np.random.choice(X_test.index, size=20, replace=False)
X_test_sample_tabfm = X_test.loc[sample_idx_tabfm].reset_index(drop=True)

t0 = time.time()
shap_values_tabfm = np.array(explainer_tabfm.shap_values(X_test_sample_tabfm, nsamples=100))

```

```
print(f"TabFM SHAP time: {(time.time() - t0) / 60:.1f} min | shape: {shap_values_tabfm.shape}")
```

```
shap.summary_plot(shap_values_tabfm[:, :, high_class_idx], X_test_sample_tabfm, plot_type="bar",
show=False)
plt.title("TabFM — Global Feature Importance (SHAP)")
plt.tight_layout()
plt.savefig("tabfm_shap_global_importance.png", dpi=150, bbox_inches="tight")
plt.show()

save_beeswarm_and_waterfall(
    "tabfm", "TabFM",
    shap_values_tabfm[:, :, high_class_idx],
    X_test_sample_tabfm,
    explainer_tabfm_kernel.expected_value[high_class_idx]
)
```

Figure 19: TabFM SHAP Computation

4.4.2 Global Feature Importance

The global importance plots rank features by their mean absolute SHAP value. CatBoost's plot covers the full test partition and stacks the contributions of the three classes, while the TabPFN and TabFM plots cover the sampled records and show the High risk class only, so the bar lengths are on different scales and the models are compared by rank rather than by magnitude.

[Insert Figure 20: CatBoost global feature importance]

Figure 20: CatBoost Global Feature Importance (SHAP)

[Insert Figure 21: TabPFN global feature importance]

Figure 21: TabPFN Global Feature Importance (SHAP)

[Insert Figure 22: TabFM global feature importance]

Figure 22: TabFM Global Feature Importance (SHAP)

Table 7: Top Ten Features by Mean Absolute SHAP Value

Rank	CatBoost	TabPFN	TabFM
1	iron	stress_level	stress_level
2	stress_level	iron	alt_liver
3	total_protein	alt_liver	total_protein
4	vitamin_d	total_protein	vitamin_d
5	alt_liver	vitamin_d	calcium
6	calcium	calcium	iron
7	manganese	manganese	manganese
8	body_water_content	chemical_use	body_water_content
9	family_hair_fall_history	family_hair_fall_history	chemical_use

The same seven continuous biomarkers (iron, stress_level, total_protein, vitamin_d, alt_liver, calcium, and manganese) formed the top seven features in all three models, and manganese was seventh in each. The models differed mainly in the order within this group. Iron was the top feature for CatBoost and second for TabPFN, but sixth for TabFM. Stress_level was first for both foundation models and second for CatBoost. At the bottom of the ranking, total_keratine and hair_texture, the two composite hair-condition scores, contributed almost nothing in all three models, and gender was last or second to last in each.

4.4.3 Direction of Feature Effects

The beeswarm plots show how feature values moved the predicted probability of High risk.

[Insert Figure 23: CatBoost beeswarm, High risk]

Figure 23: CatBoost SHAP Beeswarm (High Risk)

[Insert Figure 24: TabPFN beeswarm, High risk]

Figure 24: TabPFN SHAP Beeswarm (High Risk)

[Insert Figure 25: TabFM beeswarm, High risk]

Figure 25: TabFM SHAP Beeswarm (High Risk)

In the CatBoost plot, higher stress_level and higher alt_liver values pushed predictions toward High risk, while higher iron, total_protein, vitamin_d, calcium, manganese, and body_water_content values pushed predictions away from it. Each binary flag (stress, family_hair_fall_history, chemical_use, anemia, chronic_illness, late_night_sleep, sleep_disturbance, and water_reason) raised the High risk contribution when set to 1, and older age did the same. The TabPFN and TabFM plots showed the same direction for the leading biomarkers, with high stress_level and alt_liver raising risk and high iron, total_protein, vitamin_d, calcium, and manganese lowering it. Because the dataset was built using domain-informed relationships from dermatological literature (Section 3.2), these directions show that the models recovered the relationships built into the data, and they should not be read as new clinical evidence.

4.4.4 Patient-Level Explanations

The waterfall plots show how each feature moved one individual prediction from the baseline expected value to the final High risk score.

[Insert Figure 26: CatBoost patient waterfall]

Figure 26: CatBoost Patient-Level Waterfall Plot

[Insert Figure 27: TabPFN patient waterfall]

Figure 27: TabPFN Patient-Level Waterfall Plot

[Insert Figure 28: TabFM patient waterfall]

Figure 28: TabFM Patient-Level Waterfall Plot

Check before you paste:

1. Waterfalls show different patients. CatBoost's is the first test record, while TabPFN's and TabFM's are the first record of two different random samples. Don't compare them with each other. If you want one patient across all three, pick one record from the TabFM sample and compute its CatBoost and TabPFN values.

2. Samples are noisy. TabPFN used 30 records and TabFM 20, each with 100 kernel evaluations, so I only claim the top-seven set and the direction of effects. I did not claim rank differences such as iron being 2nd against 6th are meaningful.
3. 800-row context. I set the TabPFN reduced context to 800 rows from your Step 8b output line. Your pasted notebook never defined `X_train_small` or `tabpfn_predict_proba_small`, so confirm my reconstruction matches your cell.
4. Random samples were never seeded. The original sampling had no fixed seed, so my `np.random.seed` lines will pick different records than your figures. Screenshot your original cells for the figures, or re-run everything with the seeds.
5. Chapter 3.6 wording. It says the SHAP analyses used a "reduced training context". For TabFM that context was 500 rows, and for TabPFN 800 rows, so add those two numbers there.
6. Limitation. Kernel SHAP on 20 to 30 records is weaker than TreeExplainer on 4,322, so mention this in Chapter 5's Limitations.

Ready for 4.5 (Computational Considerations) when you are. Send the TabFM seconds-per-sample values at 100, 500, and 2,000 rows if you still have them, since I only have the 8-minute figure at 500 rows from your old draft.

4.5 Computational Considerations

Several implementation constraints arose during the experiments and shaped the experimental design. They are reported here for methodological transparency.

1. TabFM inference time and context size

TabFM (v1.0.0, PyTorch backend) was much slower than CatBoost and TabPFN at inference, and its runtime grew steeply with the size of the training context. With a 500-row context, predictions for the full test partition of 4,322 records took approximately 8 minutes. With a 2,000-row context, predictions for only 500 test records took approximately 15 minutes, which is roughly 1.8 seconds per record against about 0.11 seconds per record at 500 rows. The 2,000-row context is four times larger, but the per-record cost was roughly sixteen times higher, which indicates a strongly super-linear increase.

An attempt to give TabFM the full 17,284-row training partition as context, run in batches of 50 test records with predictions saved to a checkpoint file after each batch, did not complete within the session limits of the cloud GPU platform. TabFM was therefore evaluated with a 500-row context, the largest tested size that allowed a complete evaluation on the full test partition. The resulting asymmetry with CatBoost and TabPFN, which used all 17,284 rows, is addressed by the matched-context comparison in Section 4.3, where the 2,000-row run was in turn limited to a 500-record test subset for the same reason. The single-ensemble-member setting (`n_estimators = 1`) was used throughout because TabFM's default ensemble configuration multiplied the cost of every prediction.

Table 8: TabFM Inference Time by Training Context

Training Context	Test Records	Approximate Wall-Clock Time	Approximate Time per Record
500 rows	4,322	8 minutes	0.11 s
2,000 rows	500	15 minutes	1.8 s
17,284 rows	4,322	Did not complete	–

2. SHAP and numba incompatibility

PermutationExplainer, the method originally planned for TabPFN and TabFM, failed consistently in the notebook environments with a type-resolution error inside numba, the just-in-time compilation library that the shap package uses to accelerate its masking operations. The failure was reproduced on both Google Colaboratory and Kaggle Notebooks. It persisted after numba's JIT compilation was disabled through the

NUMBA_DISABLE_JIT environment variable and after numba and llvmlite were uninstalled and the packages reinstalled. A minimal numba function was used to confirm whether JIT compilation was working:

```
import numba

@numba.njit
def test_func(x):
    return x + 1

try:
    result = test_func(5)
    print("Numba JIT test: SUCCESS, result =", result)
except Exception as e:
    print("Numba JIT test: FAILED")
    print(e)
```

Figure 29: Numba Diagnostic Test

The problem was resolved by substituting KernelExplainer, which computes attributions by querying the model directly and does not use numba-accelerated masking. This changed only the algorithm used to estimate the Shapley values, not the underlying methodology, at the cost of longer computation per explained record. That cost is the reason the TabPFN and TabFM analyses in Section 4.4 were limited to 30 and 20 test records with a reduced training context.

3. Single-row prediction failure in TabFM

During SHAP computation, TabFM's `predict_proba` method failed when it was called with a single-row input, which KernelExplainer does routinely. This behavior is consistent with an upstream defect in the package. It was resolved by duplicating a single-row query into a two-row batch, predicting, and keeping only the first output row, which does not change the prediction for the original record. The wrapper function is given in Section 4.4.1.

4. TabPFN checkpoint loading

Loading TabPFN's pretrained weights failed on the cloud environment because recent PyTorch versions call `torch.load` with `weights_only` set to `True` by default, which rejected the checkpoint. `torch.load` was wrapped so that `weights_only` was set to `False` before the TabPFN model was created, as shown in Section 4.1.2. TabPFN's license token was read from the Kaggle secrets store instead of being written into the notebook.

5. Implications

These issues came from the recency of the tooling rather than from the models' predictive behavior. TabFM was released only shortly before this work, and its runtime scaling and single-row behavior were not evident from its documentation, so they had to be found by testing. Isolating one variable at a time (context size, batch size, backend) was necessary to tell architectural limits from implementation defects. Future users of TabFM should measure runtime at their intended context size before planning an evaluation.

I don't have the exact timings, and I found two things you should check:

1. Timing values are approximate. No file in your zip stores wall-clock times. The 8 minutes and 15 minutes come from the messages inside your notebook code (~8 min based on earlier run, ~15 min), and

the per-record values are calculated from them. Replace them with the exact times printed in your Kaggle output.

2. Old draft conflict. Your Chapter 4 draft said 0.35, 0.53, and 1.85 s per sample at 100, 500, and 2,000 rows, and "15 hours" projected for the full context. The 0.53 s figure doesn't fit an 8-minute run on 4,322 records (that is 0.11 s), so I dropped it, and I dropped the 100-row figure and the 15-hour projection because I couldn't confirm them. If you have measured values for those, send them and I'll restore them.
3. Full-context attempt. Chapter 4.1 and Chapter 3.5 must describe TabFM at 500 rows only, with no mention of the 17,284-row checkpointed run except here. Say if you want that run kept in Chapter 4 at all, since a run that never finished can raise questions.
4. Wording I can't verify. The claim that the single-row failure matches an upstream defect comes from your earlier draft. The reproduction paper [12] documents four upstream defects, but I don't know if this is one of them, so keep the sentence hedged.
5. Numbering. Table 8 and Figure 29 follow Table 7 and Figure 28 from 4.4.
6. Chapter 3.6. It says the reason for using KernelExplainer is given in Section 4.5. It is, in item 2.

Ready for 4.6 (Summary of Findings) when you are.

Yes, 4.6 is the last section of Chapter 4. The Clickbait example ends Chapter 4 at 4.2, so 4.6 is optional under your syllabus, but your accepted proposal's Chapter 4 ended with a summary, so I kept it. Delete it if your supervisor prefers the shorter layout.

4.6 Summary of Findings

This chapter reported the implementation and evaluation of CatBoost, TabPFN, and TabFM for three-tier hair fall risk classification on a 21,606-record dataset, together with a training-context scaling experiment and a SHAP explainability analysis.

On the held-out test partition of 4,322 records, the three models reached closely comparable results. Accuracy ranged from 77.88% to 78.57% and macro-F1 from 0.7715 to 0.7787, with ROC-AUC between 0.9180 and 0.9232. TabPFN scored highest on accuracy, macro-recall, macro-F1, and ROC-AUC, and CatBoost on macro-precision. TabFM reached this range with a 500-row training context, under 3% of the 17,284 records given to CatBoost and TabPFN. McNemar's test found no significant difference between any pair of models ($p = 0.9037, 0.1986, \text{ and } 0.1082$). This means no difference was detected between a grid-searched gradient-boosted model and two zero-shot foundation models that required no tuning. All three models classified the Low and High tiers more reliably than the Moderate tier, and confusions between Low and High were almost absent.

The scaling experiment showed how strongly the models depended on training volume. At 100, 500, and 2,000 rows, both foundation models scored higher than CatBoost, and the gap narrowed as the context grew, until CatBoost matched TabPFN with the full training partition. At 500 rows, CatBoost was 8.87 points below its full-data accuracy, while TabPFN was 0.92 points below its own. TabFM scored higher than TabPFN at each reduced size, but by margins that shrank to 0.23 points at 500 rows, and the 2,000-row comparison used only 500 test records, so it does not support a firm ranking between the two foundation models.

The SHAP analysis showed that iron, stress_level, total_protein, vitamin_d, alt_liver, calcium, and manganese were the seven most influential features in all three models, with differences mainly in their order. Higher

stress_level and alt_liver values, and the presence of any binary risk flag, raised the predicted High risk contribution, while higher iron, protein, vitamin D, calcium, and manganese values lowered it. The two composite hair-condition scores contributed almost nothing. These attributions reflect relationships built into the dataset, and the foundation-model attributions came from a reduced context and only 20 to 30 test records.

Finally, running the models revealed practical limits of the newer tooling. TabFM's inference time grew steeply with context size, so a full-context evaluation could not be completed, and PermutationExplainer failed because of a numba incompatibility, which led to KernelExplainer. TabFM's single-row prediction failure and TabPFN's checkpoint loading also needed workarounds.

Nothing new is claimed in this section. Every statement matches Tables 3 to 8 and Figures 12 to 28, and it wasn't written as "matched CatBoost" or "statistically equivalent", since a non-significant McNemar test doesn't prove equivalence.

Chapter 5 will need edits. Your draft Chapter 5 was written for the old results and contains claims that no longer hold:

- It says a full-context TabFM evaluation was attempted but "not feasible", which is fine, but it also says TabFM "matched or, at a 2,000-row context, exceeded" TabPFN. The 2,000-row result can't support that.
- It calls the models "statistically indistinguishable", and says the foundation-model advantage is "a genuine and reproducible property of the paradigm as a whole". Both are too strong for one dataset.
- It refers to "Section 2.1" for the dataset construction, which should be 3.2.
- Its Limitations should add the CatBoost early-stopping issue, the unequal test sets in the scaling study, and the small SHAP samples.

Ready for Chapter 5 when you are. Tell me if the file to work from is [Hairfall_Thesis_Chapter_5.pdf](#), and whether you want the same 5.1 to 5.5 structure as your draft.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

This dissertation set out to determine whether zero-shot tabular foundation models can match the predictive performance of a tuned gradient-boosted decision tree model on a multi-tier health risk classification task, and whether the resulting predictions can be traced to interpretable clinical and lifestyle features. To answer this, CatBoost, TabPFN, and TabFM were benchmarked on a 21,606-record hair fall risk dataset with three ordinal risk tiers (Low, Moderate, and High), using accuracy, macro-averaged precision, recall, and F1-score, ROC-AUC, McNemar's test, a training-context scaling experiment, and SHAP-based explanations.

On the held-out test partition of 4,322 records, the three models performed closely. Accuracy ranged from 77.88% to 78.57% and macro-F1 from 0.7715 to 0.7787. TabPFN, which required no tuning, scored highest on accuracy and macro-F1, and CatBoost, which was tuned by grid search, was within 0.06 accuracy points of it. TabFM reached this range with a 500-row training context, under 3% of the 17,284 records given to the other two models. McNemar's test found no significant difference between any pair of models at the 0.05 level. This shows that, on this dataset, no advantage of the tuned gradient-boosted model over the two zero-shot models could be detected. It does not prove that the models are equivalent.

The scaling experiment showed where the models differed most. When the training context was reduced to 100, 500, and 2,000 rows, both foundation models scored higher than CatBoost at every size, and the gap narrowed as the context grew until CatBoost matched TabPFN with the full training partition. At 500 rows, CatBoost's accuracy was 8.87 points below its full-data accuracy, while TabPFN's was 0.92 points below its own. The tuned

gradient-boosted model therefore depended more on training volume, while the foundation models retained most of their performance from small contexts. That two independently developed foundation models showed this behavior on the same dataset suggests it is not specific to one architecture, though this study tested only one dataset.

All three models classified the Low and High tiers more reliably than the Moderate tier, which borders both of the other tiers. Confusions between Low and High were almost absent, so the models' errors mostly fell between adjacent tiers.

SHAP-based explanations were produced for all three models. The same seven features (iron, stress_level, total_protein, vitamin_d, alt_liver, calcium, and manganese) were the most influential in each model, and the direction of their effects was consistent between models: higher stress_level and alt_liver values raised the predicted High risk contribution, and higher iron, total_protein, vitamin_d, calcium, and manganese values lowered it. Explainability was therefore available for the gradient-boosted model and for both foundation models, so none of the three had to be treated as an opaque black box. Because the dataset was built with domain-informed relationships, these attributions show that the models recovered the relationships built into the data. They are not new clinical findings.

5.2 Challenges Encountered During Implementation

The implementation raised several technical challenges, described in Section 4.5. The largest concerned TabFM. Its inference time grew steeply with the size of the training context, so an evaluation with the full 17,284-row context could not be completed within the session limits of the cloud platform, and the model was evaluated with a 500-row context. Its predict_proba method also failed on single-row input, which required a batch-padding workaround before SHAP values could be computed.

Two further problems came from the software environment. TabPFN's checkpoint loading failed with the installed PyTorch version until torch.load was patched. SHAP's PermutationExplainer failed on both Google Colaboratory and Kaggle Notebooks because of a numba error that persisted after JIT compilation was disabled and numba was removed and reinstalled, so KernelExplainer was used instead.

These problems came from recently released tooling and not from the modeling approaches themselves. They show that the runtime and reliability of a new package should be tested at the intended scale before an evaluation is planned around it.

5.3 Limitations of the Study

Several limitations should be noted when interpreting the results.

First, TabFM's main evaluation used a 500-row context, while CatBoost and TabPFN used 17,284 rows, so no evaluation compares all three models at full scale. The scaling experiment reduced this asymmetry, but its 2,000-row run used a 500-record test subset while the 100-row and 500-row runs used the full test partition. The 2,000-row results are therefore less precise and not directly comparable with the other sizes, and no significance tests were run at any reduced context size.

Second, the dataset was assembled from two public sources and adjusted using domain-informed relationships from dermatological literature (Section 3.2). It was not drawn from a single clinical registry, so the results measure model capability on a dataset with plausible feature-target relationships and do not validate a diagnostic tool.

Third, the SHAP analyses for TabPFN and TabFM used a reduced training context and only 30 and 20 test records, so their attributions carry more uncertainty than CatBoost's exact values on the full test partition. For that reason, only the shared set of leading features and the direction of effects were interpreted, and differences in the exact order of features were not.

Fourth, CatBoost's early stopping used the test partition as its evaluation set, so the number of trees was selected using test data. This may have favored CatBoost slightly. The finding that no difference was detected is

therefore not likely to be caused by this choice, but the comparison would be cleaner with a separate validation split.

Fifth, all results come from one train/test split with one random seed, and no confidence intervals were reported for the metrics.

Sixth, the Moderate tier was consistently the hardest to classify. An explanation based on the ordinal structure of the target was given, but the study did not test ordinal-aware models.

5.4 Future Work

Several directions follow from these findings and limitations.

- **Full-scale TabFM evaluation.** As TabFM's tooling and runtime improve, or with more powerful GPU hardware, an evaluation of TabFM with the full training context would allow all three models to be compared under identical conditions.
- **Matched-context scaling with significance testing.** The scaling experiment could be repeated on the full test partition at every context size, with several random seeds and McNemar's test at each size, to establish whether the differences between foundation models at reduced contexts are reliable.
- **More robust evaluation.** Repeated cross-validation or multiple random splits, with confidence intervals on each metric, would show how much the results depend on a single split. A separate validation partition for CatBoost's early stopping would remove the leakage noted above.
- **Ordinal modeling.** Ordinal regression or ordinal-aware classification could be compared with the standard multi-class approach, to test whether it improves accuracy on the Moderate tier.
- **Larger SHAP analyses.** Computing SHAP values for the foundation models on more test records and larger contexts would give more stable attributions and allow a closer comparison of feature rankings.
- **Ensembling.** Ensembles of the three models, including stacking approaches, could be examined to see whether they exceed the best single model.
- **Validation on real clinical data.** When ethically cleared clinical hair-loss data become available, the framework could be re-evaluated on real patients.
- **Deployment.** Because TabPFN and TabFM need their training data at inference time, distilling their behavior into a lightweight model could make low-latency deployment possible.

5.5 Final Remarks

This study compared a tuned gradient-boosted model with two zero-shot tabular foundation models on a three-tier hair fall risk task. It found no detectable difference in accuracy between them on this dataset, together with much smaller losses in accuracy for the foundation models when training data was reduced, and it produced feature-level explanations for all three models. The results come from one constructed dataset and a single data split, so they are evidence about how these models behave on structured health data and are not a clinical validation. As tabular foundation models and their software mature, they appear to be a practical alternative to gradient boosting for structured prediction tasks, including health-related ones where explainable decision support matters.

What changed from your draft Chapter 5

- TabFM described as a 500-row run. The draft's data and wording assumed this too, but its logic leaned on the 2,000-row result.
- Removed claims your results don't support. These were "statistically indistinguishable", "TabFM exceeded TabPFN at 2,000 rows", "a genuine and reproducible property of the paradigm as a whole", and the future-work item on TabFM's 2,000-row advantage. I also removed the "preliminary ensembling did not exceed the best model" sentence, since no ensembling code or output is in your zip. Ensembling is now only a future-work idea.
- Added limitations. These are the CatBoost early-stopping issue, the unequal test sets in the scaling study, the single split and seed, and the missing confidence intervals.

- Fixed cross-references. The dataset disclosure is Section 3.2, not 2.1, and the challenges cross-reference points to Section 4.5.
- No new results and no citations. The chapter only uses numbers already in Tables 3 to 8, following Syllabus 1.

Check before you submit

1. **Structure.** Your Clickbait example has only "5.1 Conclusion" and "5.2 Future Recommendations". I kept your draft's five sections. To match the example more closely, merge 5.2 and 5.3 into 5.1 and 5.5 into 5.1, and keep 5.4 as the final section. Tell me and I'll do it.
2. **Confidence intervals.** 5.3 says none were reported. If you add Wilson intervals to Table 3, delete that clause. Your Chapter 3 still promises them in 3.7 and 3.9, so either add them or remove those sentences.
3. **"Deployment" bullet.** It is from your old draft. If you plan a deployment backend, say so and I'll adjust.
4. **Remaining fixes to earlier chapters.**
 - 3.5 TabFM bullet: 500-row context, single ensemble member.
 - 3.6: 800-row and 500-row reduced contexts for SHAP.
 - 3.8: fill in the GPU, RAM, and Python version cells.
 - 2.2: the [7] and [9] sentences (CatBoost outperforming level-wise trees, "clinical ML literature stresses explainability") overclaim what those papers say.

The front matter is still to write: title page, abstract (max 500 words under Syllabus 1), acknowledgements, table of contents, and the lists of figures and tables. I can do the abstract next if you want.

References

- [1] A. Widowati, S. A. Yuniarto, and M. A. R. Syaipullah, "Explainable machine learning decision support system for multi-class hair loss severity classification," *International Journal of Technology in Education and Science*, vol. 10, no. 1, pp. 115–136, 2026.
- [2] M. Sayyad and S. Midhunchakkaravarthy, "Machine learning-based diagnostic model for Alopecia Areata prediction," *Journal of Dermatological Informatics*, vol. 15, no. 2, pp. 45–52, 2023.
- [3] C. N. V. Sai, E. Archana, B. Vivek, B. Dhanwanth, and V. K. S., "Enhancing Hairfall Prediction: A Comparative Analysis of Individual Algorithms and An Ensemble Method," *International Journal on Recent and Innovation Trends in Computing and Communication*, vol. 11, no. 6s, pp. 499–508, 2023.

- [4] S. Patel, V. Gupta, and N. Mehta, "Predictive Modeling of Hair Fall using Decision Tree and Random Forest Architectures," *EAI Endorsed Transactions on Pervasive Health and Technology*, vol. 11, pp. e1–e10, 2025.
- [5] K. Chen, L. Zhang, and R. Wang, "Predicting Hair Loss with AI: A Deep Learning Framework Combining Genetic and Scalp Health Data," in *Integrating AI, Machine Learning, and IoT in Bioinformatics: Innovations in Biotech and Medical Research*, CRC Press, pp. 100–107, 2024.
- [6] J. Anderson and M. Roberts, "Clinical feature analysis and survey-driven lifestyle metrics for predicting hair loss severity," *Applied Medical Informatics Review*, vol. 18, no. 1, pp. 78–89, 2025.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, pp. 6638–6648, 2018.
- [8] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second," *Nature Machine Intelligence*, vol. 5, no. 5, pp. 485–496, 2023.
- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.
- [10] W. Kong and A. Das, "Introducing TabFM: A zero-shot foundation model for tabular data," *Google Research Blog*, Jun. 2026. [Online]. Available: <https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/>
- [11] C. S. Shakeel, S. J. Khan, B. Chaudhry, S. F. Aijaz, and U. Hassan, "Classification Framework for Healthy Hairs and Alopecia Areata: A Machine Learning (ML) Approach," *Computational and Mathematical Methods in Medicine*, vol. 2021, Article ID 1102083, 2021.
- [12] devYRPauli, "tabfm-evaluation: Independent 3-machine evaluation of Google TabFM," *GitHub repository*, Jul. 2026. [Online]. Available: <https://github.com/devYRPauli/tabfm-evaluation>
- [13] B. Dhankour, "Hair Loss Dataset," *Kaggle*, 2023. [Online]. Available: <https://www.kaggle.com/datasets/brijaldhankour/hair-loss-dataset>
- [14] "Hair Fall Problem Survey Dataset," *Mendeley Data*, V1, 2023. [Online]. Available: <https://data.mendeley.com/datasets/g46n66frrh/1>